Exercise Sheet 05

The Export

House rules.

- One file per exercise, in `my_work/`: `ex1_split_join.py`, `ex2_write_read.py`, …
- Run everything **from the lesson folder**: `python my_work/ex3_read_csv.py`. The data lives in `data/`, and the paths on this sheet (`data/…`, `my_work/…`) only work from there.
- Type the **Given** lines first, then write the rest. Compare with the **expected output** character by character.
- Files you create go into `my_work/`, never into `data/`. The export must survive you.
- Hand in: commit and push to `lesson-05`, open a pull request, and tell Lars which part of the export annoyed you most.

Exercise 1 Split and join

8 min · `ex1_split_join.py`

> **From Mo (IT):** *"The HR system exports names as 'Surname, First'. And the product tags are one long string with semicolons. Don't ask. Can you take them apart?"*

GIVEN

```
name_field = "Robinson, Max"
tags = "umbrella;golf;compact;compact;golf"
sentence = "Keeping Britain dry since 1952"
```

(a) Split `name_field` into surname and first name, and print "First Last".

(b) Split the sentence into words. Print how many there are, then the words in reverse order, then the words joined with underscores, all lowercase.

(c) Split the tags, remove the duplicates (`set()`), sort them, and print them separated by ", ".

EXPECTED OUTPUT

```
Max Robinson
5 words
1952 since dry Britain Keeping
keeping_britain_dry_since_1952
compact, golf, umbrella
```

Hint: `last, first = name_field.split(", ")` unpacks both halves in one go. `reversed(words)` and `sorted(set(tag_list))` each give you something `" ".join()` can eat.

Exercise 2 Write a file, read it back

10 min · `ex2_write_read.py`

> **From Gerald (MD):** *"I keep my stock notes on a Post-it. Mo says a 'text file' is more robust. Prove it."*

GIVEN

```
lines = ["Umbrellas sold: 1952", "Umbrellas returned: 17", "Rainy days: 203"]
```

(a) Write the three lines into `my_work/stock_note.txt`, one per line. Use `with open(..., "w")` and remember the `"\n"`.

(b) Open the file again, read the whole thing into one string with `.read()`, print it, and print how many characters and lines it has.

(c) Open it a third time and loop over it line by line. Split each line at `": "` and print the label left-aligned in 20 characters and the number right-aligned in 6.

EXPECTED OUTPUT

```
Umbrellas sold: 1952
Umbrellas returned: 17
Rainy days: 203
Characters: 60
Lines: 3
Umbrellas sold        1952
Umbrellas returned      17
Rainy days             203
```

Hint: Each line you read still ends in `"\n"`. `.strip()` it before splitting, or the number won't line up. `print(text, end="")` stops print from adding a fourth newline.

Exercise 3 Open the export

8 min · `ex3_read_csv.py`

> **From Priya (Payroll):** *"The HR system export is in `data/staff_export_messy.csv`. Don't open it in Excel, Excel 'helps' with the dates. Just tell me what's in it."*

GIVEN

```
import csv
```

(a) Open the file with `csv.DictReader` and print the column names (`reader.fieldnames`).

(b) Put all rows in a list. Print how many there are, then for the first three rows print the ref, the name and the salary, separated by " | ".

(c) Print the first row's salary with `repr()` so you can see exactly what text arrived.

EXPECTED OUTPUT

```
Columns: ['Employee Ref', 'Name', 'Dept', 'Hours', 'Salary', 'Bonus', 'Gender', 'Start']
Rows: 40
 EMP1092  | Robinson, Max | £41,900
emp1178 | Shaw, Imogen | 26550
emp1105 | ROBERTS, AMARA | 24950
First salary as text: '£41,900'
```

Hint: Always open CSV files with `newline=""` and `encoding="utf-8"`, or the £ signs and line endings go wrong on Windows. Yes, the spaces around `EMP1092` are really in the file.

Exercise 4 Clean it up

12 min · `ex4_clean.py`

> **From Priya:** *"Refs in three different styles, names SHOUTED or whispered, salaries with £ or without, hours with 'hrs' on the end. Every. Single. Month."*

GIVEN

```
import csv
```

(a) Write three functions: `clean_ref(text)` (strip, uppercase, add "EMP" if it's missing), `clean_name(text)` (split at the comma, strip both halves, Title Case, "First Last") and `clean_money(text)` (strip, remove £ and commas, empty text means `0.0`, otherwise a float).

(b) Read the export and, for the first 8 rows, print the clean ref, the name left-aligned in 18, the hours as a number right-aligned in 6, and the salary as money right-aligned in 11.

EXPECTED OUTPUT

```
EMP1092  Max Robinson        37.5   £41,900.00
EMP1178  Imogen Shaw         37.5   £26,550.00
EMP1105  Amara Roberts       37.5   £24,950.00
EMP1180  Tara Wood           37.5   £29,000.00
EMP1189  Niamh Wright        37.5   £31,750.00
EMP1082  Ibrahim Hussain     37.5   £37,650.00
EMP1037  Will Wood           37.5   £25,900.00
EMP1199  Ben Walker          22.5   £37,950.00
```

Hint: Hours: `float(row["Hours"].replace("hrs", "").strip())`. One name has spaces *inside* the comma ("Jones , Liam"), so strip *after* splitting. Make the money text first, then right-align it.

Exercise 5 Write the clean file

12 min · `ex5_write_csv.py`

> **From Siobhan (Reward):** *"Lovely. Now can I have that as a file I can actually open? Columns: employee_id, name, gender, hours, salary, bonus. Gender as one letter, M or F, whatever the system felt like that day."*

GIVEN

```
import csv
```

(a) Reuse your three cleaning functions (copy them in) and add `clean_gender(text)`: the first letter, uppercase.

(b) Build a list of dicts, one per row, with exactly the six keys Siobhan asked for.

(c) Write it to `my_work/staff_clean.csv` with `csv.DictWriter`, header included. Print how many rows you wrote.

(d) Open your new file as plain text and print its first four lines, to prove it's real.

(e) Count the women in the export and print the number.

EXPECTED OUTPUT

```
Wrote 40 rows
employee_id,name,gender,hours,salary,bonus
EMP1092,Max Robinson,M,37.5,41900.0,0.0
EMP1178,Imogen Shaw,F,37.5,26550.0,0.0
EMP1105,Amara Roberts,F,37.5,24950.0,0.0
Women in the export: 15
```

Hint: `fieldnames=list(clean_rows[0])` uses the keys of the first dict as the header. Open the output with `newline=""` too, or Windows gives you a blank line after every row.

Exercise 6 Department totals

10 min · `ex6_department_totals.py`

> **From Gerald:** *"The board asked which department costs the most. I said 'Production, probably?' They wrote that down. Please make it true or false by Thursday."*

GIVEN

```
import csv
```

(a) Read `data/brightwell_payroll.csv` (the clean one, 200 employees) and build two dicts: total salary per department and headcount per department.

(b) Print a table, departments sorted alphabetically: name in 12, staff in 6, total in 14 with thousands separators, average in 12 with no decimals.

(c) Print the department with the biggest wage bill.

EXPECTED OUTPUT

```
Department   Staff         Total     Average
Design          15       495,250      33,017
Finance         18       585,850      32,547
HR              12       397,550      33,129
IT              20       808,300      40,415
Marketing       15       463,650      30,910
Production      60     1,748,900      29,148
Sales           35     1,129,250      32,264
Warehouse       25       771,100      30,844
Biggest wage bill: Production
```

Hint: `totals.get(dept, 0) + salary` works whether the department is already in the dict or not. `max(totals, key=totals.get)` gives the key with the largest value.

Exercise 7 Files about files STRETCH

12 min · `ex7_pathlib_report.py`

> **From Mo:** *"Which files are even in that data folder? And can the answer be a file too, so I can email it to Gerald instead of explaining it?"*

GIVEN

```
import csv
from pathlib import Path
```

(a) Use `Path("data").glob("*.csv")` to find every CSV in the data folder. For each one (sorted), print its name left-aligned in 28 and how many data rows it has (lines minus the header) right-aligned in 5.

(b) Find the highest-paid employee in the payroll file.

(c) Write a three-line report to `my_work/report.txt`: a title, "Highest paid: First Last (Department)", and the salary. Then print "Wrote report.txt with 3 lines" and the report itself.

EXPECTED OUTPUT

```
brightwell_payroll.csv        200 rows
staff_export_messy.csv         40 rows
Wrote report.txt with 3 lines
BRIGHTWELL PAYROLL: QUICK FACTS
Highest paid: Lucy Thompson (Finance)
Salary: £68,750
```

Hint: A `Path` can be opened like a filename, and `Path("my_work") / "report.txt"` builds a path with a slash. `report.read_text()` reads a whole file in one call. Print `report.name`, not `report`: Windows shows paths with backslashes.

Bug hunt Dave's Leftovers #5

15 min · `bughunt/export_bug.py`

Dave's monthly stats script. Priya says it "used to work on his machine". There are **three** bugs: one crashes before a single row is read, one gives a wrong count without crashing, and one loses the pence.

EXPECTED OUTPUT ONCE FIXED

```
Employees: 200
Full-time: 174
```

```
Total salary: £6,399,850
Average salary: £31,999.25
```

Commit each fix separately, with a message explaining what was wrong.